delegate-backup Plugin - Full Specification

A Claude Code plugin that keeps a delegation pipeline running when implementers run out of quota. Each lane has an ordered chain of implementers; the plugin walks the lane one position down the chain per exhaustion, and schedules its return to the primary for when the provider's window resets.

How to use this document: drop this file into any capable AI assistant with the instruction "build this". It is written to be sufficient on its own - schema, commands, exit codes, guarantees, and the reasoning behind each decision.

Version 2.0.0 - fallback chains. Version 1.x allowed a single backup per lane, which meant a lane whose primary and backup were both exhausted had nowhere left to go. 2.0 replaces that with an arbitrary-length chain and one rule that makes the dead end unreachable: end every chain with a free, unmetered model. Companion to code-pro 2.1.0.

1. Plugin overview

Name: delegate-backup

Author: Somar

Version: 2.0.0

Purpose: Survive quota exhaustion in a multi-provider delegation setup without manual reconfiguration, without stranding a lane on a fallback after the window resets, and without ever reaching a state where no implementer is available.

Target platform: Claude Code (.claude-plugin/plugin.json + skills/ + commands/ + scripts/)

Distribution: github.com/SomarRezq/pro-marketplace - /plugin install delegate-backup@pro-marketplace

Hard dependency: a delegate-skills fleet config (github.com/amElnagdy/delegate-skills). Soft dependency: the scheduled-tasks tools - without them a swap still works, but must be reversed manually with resolve.

2. The problem it solves

Subscription coding CLIs are priced for human-paced interactive use. Sustained agentic work exhausts them on very different timescales - hours for some, days for others - and each provider has its own reset window.

A delegation pipeline's usual degradation ladder does not help. It resolves an implementer as usable when the binary is on PATH and its relay is installed, evaluated once at preflight. An implementer that is installed, authenticated, and simply out of quota passes that check and then fails its dispatch.

Version 1.x closed most of that gap but kept one hole: with a single backup per lane, losing both the primary and the backup left the lane dead. In practice that is not an edge case - two providers exhausting inside the same week is ordinary at high volume.

3. The two decisions that shape everything

3.1 Chains end in a free model

A lane's fallbacks are an ordered array. Position 0 mirrors the lane's primary in config.json; each apply advances one position. Because the final position is a free, unmetered model, the chain cannot be walked off the end - the pipeline degrades in quality rather than stopping.

Two such models were verified writing code that executes with passing assertions, in roughly 18 seconds, at zero cost and requiring no additional credential: opencode/nemotron-3-ultra-free and opencode/hy3-free. They are a floor, not a substitute - expect to review their output more carefully.

3.2 Exhaustion is never classified automatically

The plugin never decides on its own that a provider is exhausted. It is invoked with an explicit lane and window, after a human or an orchestrator has read the actual error.

This is deliberate, and it is the most important thing to preserve if this specification is ever reimplemented. Provider error messages are not trustworthy classifiers, and there are two documented counter-examples on a single provider:

- Z.AI error 1113, "Insufficient balance or no resource package", is a configuration fault. Plan quota does not fall through to account balance, so this means the call did not qualify as plan usage - wrong endpoint, or a model outside the plan.
- "Rate limit reached for requests" is a per-minute throttle, not a spent budget. The correct response is to retry, not to swap.

An automatic classifier keying on either phrase would silently move work onto a fallback in order to route around a bug the user needed to see. A fallback that hides a misconfiguration is worse than a hard failure. So the split is: judgment stays with the human or orchestrator; bookkeeping is automated.

4. File layout and state

delegate-skills' config.json is never extended with new keys. It validates dials strictly and owns that document, so every piece of state this plugin needs lives in a sidecar. A delegate-skills upgrade can therefore never collide with this plugin.

Path	Owner	Contents
~/.config/delegate-skills/config.json	delegate-skills	The fleet. This plugin only ever rewrites the value of an existing lane.

~/.config/delegate-skills/lane-backups.json	delegate-backup	chains (policy), active (current position per lane), history (audit trail).
~/.claude/scheduled-tasks/delegate-restore-<lane>/	scheduled-tasks	One one-shot restore task per swapped lane.

If XDG_CONFIG_HOME is set, both config files live under $XDG_CONFIG_HOME/delegate-skills/ instead. This mirrors delegate-skills' own resolution order.

4.1 Sidecar schema (delegate-backups.v2)

```
{
 "version": "delegate-backups.v2",
 "chains": {
  "feature": [
   { "implementer": "opencode", "model": "zai-coding-plan/glm-5.3-flash", "variant": "low" },
   { "implementer": "codex", "effort": "medium" },
   { "implementer": "opencode", "model": "opencode/nemotron-3-ultra-free" }
  ]
 },
 "active": {
  "feature": {
   "original":  { ...the lane before we ever touched it... },
   "position":  2,
   "wrote":    { ...what we last wrote into config.json... },
   "appliedAt": "2026-08-30T01:09:12+02:00",
   "expiresAt": "2026-09-02T05:29:47+02:00",
   "reason":   "GLM: Weekly/Monthly Limit Exhausted",
   "taskId":   "delegate-restore-feature"
  }
 },
 "history": []
}
```

chains is policy and is edited by the user; position 0 must mirror the lane's primary. active and history are written by the script. original is captured on the first swap and preserved across every subsequent advance, so a restore always returns to the user's own configuration rather than to an intermediate position.

4.2 Migration from v1

A delegate-backups.v1 sidecar is upgraded in memory on read and persisted on the next write. Each single backup becomes a two-element chain: the lane's current primary from config.json at position 0, and the old backup at position 1. A v1 active entry, which had no position, is recorded as position

1. Existing behaviour is preserved exactly; adding a free model to the end of each chain is what buys the v2 guarantee.

5. Commands

Command	What it does
apply --lane <n> --until <win> [--reason ..] [--dry-run]	Advance the lane one position down its chain, record how to undo it, and print the scheduled-task spec to create. Reports movedTo and fallbacksRemaining.
resolve [--lane <n>] [--all] [--force]	Return lanes to position 0. --lane restores that lane now (the task fired). --all restores every lane whose window has passed.
status [--json]	Show each chain with the live position marked, plus any active fallback.

--until accepts a duration (71h37m, 5h, 3d) or an ISO 8601 timestamp. Both forms occur in practice: Codex and Antigravity report a duration, Z.AI reports an absolute timestamp. Use whichever the provider gave.

When fallbacksRemaining reaches 0 the output carries a warning field: the lane is on its last entry. The caller should surface that to the user.

5.1 Exit codes

Code	Meaning	Caller's move
0	Advanced one position	Create the printed scheduled task; report the new position
1	Usage or config error	Fix the invocation
2	No chain configured for that lane	Tell the user; offer to add one
3	resolve found nothing due	Normal - not an error
4	Chain exhausted - every entry spent	Stop. Report it and let the user decide

Exit 4 should be unreachable in a healthy setup, because every chain ends in a free model. Reaching it means the chain is misconfigured. The caller must never work around it by editing the fleet config by hand or substituting an implementer that is not in the chain.

6. Guarantees

Guarantee	Mechanism
Cannot run out of fallbacks	Chains are arbitrary length and end in a free, unmetered model.
Cannot strand a lane	Every swap records expiresAt. resolve --all restores anything overdue even if the scheduled task was lost or never fired.
Cannot clobber your edits	A restore only fires if the lane still holds exactly what the swap wrote. A hand-edit since the swap wins, and the entry is abandoned.
Restores are deterministic	original is captured on the first swap and preserved across every advance.
Runs exactly once	Restore tasks use fireAt, which auto-disables after firing.
Survives downtime	A task due while the app was closed runs on next launch.
No half-written config	Writes go to a temp file and are renamed into place.
Never invisible	code-pro preflight prints every active fallback and its chain position.

7. The restore lifecycle

apply does not create the scheduled task itself - creating and deleting tasks are tools only the orchestrator can call. The script prints a complete task spec and the skill creates it verbatim.

The task prompt is deliberately reduced to a single command. A scheduled run starts with no memory of the conversation that created it, so there must be nothing to interpret:

Run this exact command and report its output verbatim:

node "<plugin>/scripts/backup.mjs" resolve --lane ui

Then delete this scheduled task (taskId: delegate-restore-ui). If deleting is not possible, stop - the task is one-shot and will not fire again.

Self-deletion is tidiness only, never correctness: fireAt already guarantees the task cannot fire twice. If the delete call is unavailable or needs an approval nobody is present to give, the task remains disabled and harmless, and the next resolve prunes its directory.

Restore always returns the lane to position 0, never to an intermediate position. If position 0 is still exhausted, the next dispatch fails and apply advances again - one wasted call, in exchange for a state machine with no per-position cooldown bookkeeping. That trade is deliberate: per-position tracking is where this design becomes complex and subtly wrong.

8. Building a chain

- End it with a free, unmetered model. This is the rule that makes exit 4 unreachable.
- Never put a shell-incapable implementer on tests or qa. Antigravity soft-denies every shell call in the non-interactive mode the relays use, so those lanes would report success without executing anything - a correctness failure, not an efficiency one.
- Change provider at every position, or the fallback shares the outage. All Z.AI coding-plan models share a single bucket: glm-5.3, glm-5.3-flash, glm-5-turbo and glm-4.7 exhaust together and reset together, so none can back another.
- For review, avoid the provider that implemented the code, or the review stops being an independent second opinion.

9. Known provider messages

Implementer	Message	Exhaustion?	Window
codex	You have hit your usage limit	Yes	retry time
agy	Individual quota reached. Resets in XhYmZs	Yes	duration
opencode (Z.AI plan)	Weekly/Monthly Limit Exhausted. Your limit will reset at <timestamp>	Yes	timestamp
copilot	You have exceeded your premium request allowance / HTTP 402	Yes	none given
opencode (Z.AI)	Insufficient balance or no resource package (1113)	NO - configuration	-
opencode (Z.AI free)	Rate limit reached for requests	NO - throttle, retry	-

10. Integration with code-pro

code-pro 2.1.0 treats this plugin as a soft dependency. Its preflight reads lane-backups.json directly rather than shelling out, so a missing plugin costs one informational line instead of a failure. When fallbacks are active, preflight gains:

```
Lane backups (active)
  OK   ui       on opencode  [2/2] -> back to agy       in 71h 37m
```

The [position/last] marker matters: it tells the reader how much fallback headroom is left before the chain bottoms out. An entry marked OVERDUE means a scheduled restore never fired and resolve --all should be run. Running resolve --all at the start of a session is the recommended safety net.

11. Build instructions for the AI reading this file

Produce a Claude Code plugin with this layout:

- .claude-plugin/plugin.json - name, description, version 2.0.0, author
- scripts/backup.mjs - all logic; Node built-ins only, no dependency to install
- skills/delegate-backup/SKILL.md - when to swap, how to tell exhaustion from misconfiguration and from throttling, exit-code handling
- commands/delegate-backup.md - status + resolve --all, and a summary for the user
- lane-backups.example.json - a copyable starting sidecar, chains ending in free models
- README.md - install, design, exit codes, chain-building rules

Hard requirements: never write an unknown key into config.json; never restore a lane whose current value differs from what was written; preserve original across every advance; write config atomically; migrate v1 sidecars without changing behaviour; and treat exhaustion classification as the caller's job, never the script's.